
SIMATS ENGINEERING

Department of Computer Science Engineering

CSA0609 — Design and Analysis of Algorithms

Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems

Particular	Details
Student Name	ADITHYA R
Register / Student ID	192311369
Section / Batch	2023-2027
Programme	Bachelor of Engineering
Course Code & Course Name	CSA0609-Design and Analysis of Algorithms
Academic Year / Batch	2026-Regular
Faculty Name	Dr.P.Solainayagi
Assignment Title	Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems
Date of Issue	01-09-2026
Date of Submission	02-09-2025
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education: Develops analytical thinking, programming, and computational problem-solving skills. SDG 9 – Industry, Innovation and Infrastructure – Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.
Industry / Societal Relevance	Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.

Problem Statement, Objectives & Given Data

Problem Statement

Given a set of 10 two-dimensional points, apply the Brute Force technique to solve the Closest Pair and Convex Hull problems. The pair of points having the minimum Euclidean distance must be determined by considering every possible pair and computing the distance using the Euclidean distance formula. The closest pair must be identified and justified, a suitable algorithm/pseudocode developed, and the number of distance comparisons analysed. Using the same set of points, the Convex Hull must be determined by examining the relationships between points using the orientation test, identifying all hull vertices, showing the major intermediate calculations, and providing a graphical representation of the resulting hull. A suitable Brute Force algorithm/pseudocode for the Convex Hull must be developed and its time and space complexity analysed. Finally, the two Brute Force approaches are compared in terms of strategy, computational cost as n increases, limitations for large datasets, and possible improvements using efficient algorithms (Divide-and-Conquer and Hybrid methods).

Objectives

- Compute the Closest Pair of points among the given 10 points using the Brute Force method, evaluating all $C(10,2) = 45$ distances.
- Compute the Convex Hull of the given points using the Brute Force orientation-test method.
- Develop, implement, and test correct, compilable C programs for both problems.
- Analyse the time and space complexity of both Brute Force algorithms.
- Design Divide-and-Conquer and Hybrid variants and compare all four approaches experimentally on a real-world coordinate dataset.

Given Input Data — Set of 10 Points (P)

Point	Coordinates (x, y)	Point	Coordinates (x, y)
P1	(2, 3)	P6	(3, 11)
P2	(5, 8)	P7	(15, 6)
P3	(9, 4)	P8	(10, 14)
P4	(12, 10)	P9	(6, 12)
P5	(7, 2)	P10	(1, 7)

Table 2.1 — Input point set $P = \{P1 \dots P10\}$ used throughout this study.

Problem Analysis & Governing Formulas

Analysis

The Closest Pair problem asks for the two points in P with the smallest Euclidean separation. The naive (Brute Force) strategy checks every unordered pair exactly once, so the number of distance evaluations equals the number of 2-combinations of n points, $C(n,2)$. For $n = 10$ this yields exactly 45 comparisons. The Convex Hull problem asks for the smallest convex polygon enclosing all points in P . The Brute Force strategy tests every ordered pair (P_i, P_j) as a candidate hull edge: the edge belongs to the hull if and only if every other point lies strictly on one side of the line through P_i and P_j , verified using the orientation (cross-product) test. Both problems therefore reduce to systematically evaluating a geometric predicate over all pairs/triples of points.

Euclidean Distance Formula (Closest Pair)

$$d(A, B) = \sqrt{[(B_x - A_x)^2 + (B_y - A_y)^2]}$$

Number of Pairwise Comparisons (Brute Force Closest Pair)

$$C(n, 2) = n(n - 1) / 2 \Rightarrow C(10, 2) = (10 \times 9) / 2 = 45$$

Orientation Test (Convex Hull)

$$\text{cross}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x)$$

If $\text{cross} > 0$, point C lies to the left of the directed line $A \rightarrow B$ (counter-clockwise turn); if $\text{cross} < 0$, C lies to the right (clockwise turn); if $\text{cross} = 0$, the three points are collinear. Edge (A,B) is a hull edge exactly when all other points give the same sign.

Complexity Notation Used in this Report

- Brute Force Closest Pair — evaluates all $C(n,2)$ pairs $\Rightarrow O(n^2)$ time, $O(1)$ extra space.
- Brute Force Convex Hull — evaluates all $O(n^2)$ candidate edges, each checked against the remaining $O(n)$ points $\Rightarrow O(n^3)$ time, $O(1)$ extra space.
- Divide-and-Conquer Closest Pair — $O(n \log n)$ time, $O(n)$ space.
- Divide-and-Conquer Convex Hull (Quickhull) — $O(n \log n)$ average, $O(n^2)$ worst case.

Closest Pair — Brute Force Algorithm & Pseudocode

Algorithm — Steps

1. Read the n input points into an array $P[0 \dots n-1]$.
2. Initialise $\text{minDist} \leftarrow \infty$.
3. For every index i from 0 to $n-2$, and every index j from $i+1$ to $n-1$, compute $d = \text{distance}(P[i], P[j])$.
4. If $d < \text{minDist}$, update $\text{minDist} \leftarrow d$ and remember the pair (i, j) .
5. After all $C(n,2)$ pairs are examined, report the remembered pair and minDist as the closest pair and its distance.

Pseudocode

ALGORITHM BruteForceClosestPair($P[0..n-1]$)

```
-----
minDist <- INFINITY
minPair <- (NULL, NULL)
count   <- 0

for i <- 0 to n-2 do
  for j <- i+1 to n-1 do
    d <- sqrt( (P[j].x - P[i].x)^2 + (P[j].y - P[i].y)^2 )
    count <- count + 1
    if d < minDist then
      minDist <- d
      minPair <- (P[i], P[j])
    end if
  end for
end for

return (minPair, minDist, count)  // count = C(n,2)
-----
```

Correctness follows directly from exhaustive enumeration: because every one of the $C(n,2)$ unordered pairs is examined exactly once, the pair yielding the global minimum distance is guaranteed to be found.

Closest Pair — Complete C Program

```
/* Closest Pair of Points - Brute Force Approach
   Course: CSA06 - Design and Analysis of Algorithms */
#include <stdio.h>
#include <math.h>
#include <float.h>
#define N 10
typedef struct { int x, y; } Point;

double distance(Point a, Point b) {
    return sqrt((double)(a.x - b.x) * (a.x - b.x) +
                (double)(a.y - b.y) * (a.y - b.y));
}

int main() {
    Point pts[N] = {
        {2, 3}, {5, 8}, {9, 4}, {12, 10}, {7, 2},
        {3, 11}, {15, 6}, {10, 14}, {6, 12}, {1, 7}
    };
    double minDist = DBL_MAX;
    int minI = 0, minJ = 1, comparisons = 0;

    printf("=====\n");
    printf(" CLOSEST PAIR OF POINTS - BRUTE FORCE (n = %d)\n", N);
    printf("=====\n\n");
    printf("Input Points:\n");
    for (int i = 0; i < N; i++)
        printf(" P%d = (%d, %d)\n", i + 1, pts[i].x, pts[i].y);
    printf("\nPairwise Distance Table:\n");
    printf("-----\n");
    printf(" Pair      Points      Distance\n");
    printf("-----\n");
    for (int i = 0; i < N - 1; i++) {
        for (int j = i + 1; j < N; j++) {
            double d = distance(pts[i], pts[j]);
            comparisons++;
            printf(" (P%-2d,P%-2d) (%2d,%2d)-(%2d,%2d)   %.4f\n",
                i + 1, j + 1, pts[i].x, pts[i].y, pts[j].x, pts[j].y, d);
            if (d < minDist) { minDist = d; minI = i; minJ = j; }
        }
    }
    printf("-----\n");
    printf("\nTotal distance comparisons performed = %d\n", comparisons);
    printf("Expected C(%d,2) = %d\n", N, N * (N - 1) / 2);
    printf("\n=====\n");
    printf(" RESULT\n");
    printf("=====\n");
    printf("Closest Pair : P%d(%d,%d) and P%d(%d,%d)\n",
        minI + 1, pts[minI].x, pts[minI].y, minJ + 1, pts[minJ].x, pts[minJ].y);
    printf("Minimum Distance = %.4f units\n", minDist);
    printf("=====\n");
    return 0;
}
```

Closest Pair — Program Output

The program above was compiled with GCC and executed. The terminal capture below shows the actual output produced (the middle rows of the 45-row distance table are omitted here purely for page space — the complete table is reproduced as Table 7.1 on Page 7).

```
closest_pair.c -o closest_pair -lm && ./closest_pair

=====
CLOSEST PAIR OF POINTS - BRUTE FORCE (n = 10)
=====

Input Points:
P1 = (2, 3)
P2 = (5, 8)
P3 = (9, 4)
P4 = (12, 10)
P5 = (7, 2)
P6 = (3, 11)
P7 = (15, 6)
P8 = (10, 14)
P9 = (6, 12)
P10 = (1, 7)

Pairwise Distance Table:
-----
Pair          Points          Distance
-----
(P1 ,P2 )    ( 2, 3)-( 5, 8)         5.8310
(P1 ,P3 )    ( 2, 3)-( 9, 4)         7.0711
(P1 ,P4 )    ( 2, 3)-(12,10)        12.2066
(P1 ,P5 )    ( 2, 3)-( 7, 2)         5.0990
(P1 ,P6 )    ( 2, 3)-( 3,11)         8.0623
(P1 ,P7 )    ( 2, 3)-(15, 6)        13.3417
(P1 ,P8 )    ( 2, 3)-(10,14)        13.6015
(P1 ,P9 )    ( 2, 3)-( 6,12)         9.8489
(P1 ,P10)    ( 2, 3)-( 1, 7)          4.1231
(P2 ,P3 )    ( 5, 8)-( 9, 4)         5.6569
(P2 ,P4 )    ( 5, 8)-(12,10)        7.2801
(P2 ,P5 )    ( 5, 8)-( 7, 2)         6.3246
(P2 ,P6 )    ( 5, 8)-( 3,11)         3.6056
(P2 ,P7 )    ( 5, 8)-(15, 6)        10.1980
... (25 more rows omitted for space) ...
(P7 ,P8 )    (15, 6)-(10,14)         9.4340
(P7 ,P9 )    (15, 6)-( 6,12)        10.8167
(P7 ,P10)    (15, 6)-( 1, 7)         14.0357
(P8 ,P9 )    (10,14)-( 6,12)         4.4721
(P8 ,P10)    (10,14)-( 1, 7)        11.4018
(P9 ,P10)    ( 6,12)-( 1, 7)         7.0711
-----

Total distance comparisons performed = 45
Expected C(10,2) = 45

=====
RESULT
=====
Closest Pair : P3(9,4) and P5(7,2)
Minimum Distance = 2.8284 units
=====
```

Figure 6.1 — Actual terminal output of `closest_pair.c` (`gcc -o closest_pair closest_pair.c -lm`).

Closest Pair — Complete Pairwise Calculations

#	Pair	Points	Distance	#	Pair	Points	Distance
1	P1-P2	(2,3)–(5,8)	5.8310	24	P3-P10	(9,4)–(1,7)	8.5440
2	P1-P3	(2,3)–(9,4)	7.0711	25	P4-P5	(12,10)–(7,2)	9.4340
3	P1-P4	(2,3)–(12,10)	12.2066	26	P4-P6	(12,10)–(3,11)	9.0554
4	P1-P5	(2,3)–(7,2)	5.0990	27	P4-P7	(12,10)–(15,6)	5.0000
5	P1-P6	(2,3)–(3,11)	8.0623	28	P4-P8	(12,10)–(10,14)	4.4721
6	P1-P7	(2,3)–(15,6)	13.3417	29	P4-P9	(12,10)–(6,12)	6.3246
7	P1-P8	(2,3)–(10,14)	13.6015	30	P4-P10	(12,10)–(1,7)	11.4018
8	P1-P9	(2,3)–(6,12)	9.8489	31	P5-P6	(7,2)–(3,11)	9.8489
9	P1-P10	(2,3)–(1,7)	4.1231	32	P5-P7	(7,2)–(15,6)	8.9443
10	P2-P3	(5,8)–(9,4)	5.6569	33	P5-P8	(7,2)–(10,14)	12.3693
11	P2-P4	(5,8)–(12,10)	7.2801	34	P5-P9	(7,2)–(6,12)	10.0499
12	P2-P5	(5,8)–(7,2)	6.3246	35	P5-P10	(7,2)–(1,7)	7.8102
13	P2-P6	(5,8)–(3,11)	3.6056	36	P6-P7	(3,11)–(15,6)	13.0000
14	P2-P7	(5,8)–(15,6)	10.1980	37	P6-P8	(3,11)–(10,14)	7.6158
15	P2-P8	(5,8)–(10,14)	7.8102	38	P6-P9	(3,11)–(6,12)	3.1623
16	P2-P9	(5,8)–(6,12)	4.1231	39	P6-P10	(3,11)–(1,7)	4.4721
17	P2-P10	(5,8)–(1,7)	4.1231	40	P7-P8	(15,6)–(10,14)	9.4340
18	P3-P4	(9,4)–(12,10)	6.7082	41	P7-P9	(15,6)–(6,12)	10.8167
19	P3-P5	(9,4)–(7,2)	2.8284	42	P7-P10	(15,6)–(1,7)	14.0357
20	P3-P6	(9,4)–(3,11)	9.2195	43	P8-P9	(10,14)–(6,12)	4.4721
21	P3-P7	(9,4)–(15,6)	6.3246	44	P8-P10	(10,14)–(1,7)	11.4018
22	P3-P8	(9,4)–(10,14)	10.0499	45	P9-P10	(6,12)–(1,7)	7.0711
23	P3-P9	(9,4)–(6,12)	8.5440				

Table 7.1 — All 45 pairwise Euclidean distances ($C(10,2) = 45$). The minimum-distance row is highlighted in green.

Result: the minimum distance found is 2.8284 units, between P3(9,4) and P5(7,2) — confirmed identically by the program output on Page 6 and by the divide-and-conquer/hybrid implementations on Pages 15–16.

Convex Hull — Brute Force Explanation

Core Idea

The Brute Force Convex Hull algorithm is based on a simple geometric fact: a directed edge (P_i, P_j) between two points of the set is an edge of the convex hull if and only if all remaining $n-2$ points lie strictly on the same side of the line through P_i and P_j . This "same-side" test is implemented using the orientation (signed cross-product) formula introduced on Page 3.

How the Method Works

- Every ordered pair (P_i, P_j) , $i \neq j$, is treated as a candidate hull edge — there are $n(n-1)$ such ordered candidates.
- For each candidate edge, every other point P_k ($k \neq i, j$) is classified using $\text{cross}(P_i, P_j, P_k)$: positive, negative, or zero (collinear).
- If all non-zero classifications share the same sign (all points lie on one side), the edge (P_i, P_j) is a genuine hull edge, and both P_i and P_j are marked as hull vertices.
- After all candidate edges are tested, the set of marked vertices, arranged in order, forms the Convex Hull boundary.

Why This is a Valid Brute Force Strategy

The method makes no assumption about the shape or ordering of the input; it exhaustively tests every possible edge against every possible witness point. This exhaustiveness is exactly what classifies it as "brute force" — correctness is guaranteed by complete enumeration, at the cost of examining $O(n^2)$ candidate edges, each requiring an $O(n)$ side-test, for an overall $O(n^3)$ running time.

Relation to the Given Point Set

Applying this method to $P = \{P_1 \dots P_{10}\}$ (Page 2) identifies exactly six points lying on the outer boundary and four interior points. The full edge-by-edge verification is presented on Page 9 (algorithm), Page 10 (code), Page 11 (output) and Page 13 (intermediate orientation calculations).

Convex Hull — Brute Force Algorithm & Pseudocode

Algorithm — Steps

6. Read the n input points into an array $P[0 \dots n-1]$.
7. For every ordered pair (i, j) , $i \neq j$, initialise $\text{pos} \leftarrow 0$, $\text{neg} \leftarrow 0$.
8. For every other index k ($k \neq i, j$), compute $\text{cross} = \text{orientation}(P[i], P[j], P[k])$; increment pos if $\text{cross} > 0$, neg if $\text{cross} < 0$.
9. If $\text{pos} = 0$ or $\text{neg} = 0$ (all remaining points on one side), mark $P[i]$ and $P[j]$ as hull vertices.
10. After all ordered pairs are processed, output every point marked as a hull vertex.

Pseudocode

ALGORITHM BruteForceConvexHull($P[0..n-1]$)

```
-----
isHull[0..n-1] <- false

for i <- 0 to n-1 do
  for j <- 0 to n-1, j != i do
    pos <- 0 ; neg <- 0
    for k <- 0 to n-1, k != i, k != j do
      cross <- (P[j].x-P[i].x)*(P[k].y-P[i].y)
        - (P[j].y-P[i].y)*(P[k].x-P[i].x)
      if cross > 0 then pos <- pos + 1
      else if cross < 0 then neg <- neg + 1
    end for
    if pos = 0 OR neg = 0 then
      isHull[i] <- true
      isHull[j] <- true
    end if
  end for
end for

return { P[i] : isHull[i] = true }
-----
```

Convex Hull — Complete C Program

```
/* Convex Hull - Brute Force Approach (Orientation Method)
   Course: CSA06 - Design and Analysis of Algorithms */
#include <stdio.h>
#include <math.h>
#define N 10
typedef struct { int x, y; } Point;

/* Returns (Bx-Ax)(Cy-Ay) - (By-Ay)(Cx-Ax) */
long orientation(Point A, Point B, Point C) {
    return (long)(B.x - A.x) * (C.y - A.y) - (long)(B.y - A.y) * (C.x - A.x);
}

int main() {
    Point pts[N] = {
        {2, 3}, {5, 8}, {9, 4}, {12, 10}, {7, 2},
        {3, 11}, {15, 6}, {10, 14}, {6, 12}, {1, 7}
    };
    int isHull[N] = {0}, edgeChecks = 0;

    printf("=====\n");
    printf(" CONVEX HULL - BRUTE FORCE / ORIENTATION METHOD (n=%d)\n", N);
    printf("=====\n\n");
    printf("Input Points:\n");
    for (int i = 0; i < N; i++)
        printf(" P%d = (%d, %d)\n", i + 1, pts[i].x, pts[i].y);
    printf("\nChecking every ordered pair (Pi, Pj) as a candidate edge:\n");
    printf("A candidate edge is a HULL EDGE if all other points lie\n");
    printf("strictly on ONE side of the line (all cross products\n");
    printf("have the same sign).\n\n");
    printf("-----\n");
    printf(" Edge      Pos Neg  Hull Edge?\n");
    printf("-----\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i == j) continue;
            int pos = 0, neg = 0;
            for (int k = 0; k < N; k++) {
                if (k == i || k == j) continue;
                edgeChecks++;
                long cr = orientation(pts[i], pts[j], pts[k]);
                if (cr > 0) pos++; else if (cr < 0) neg++;
            }
            int isEdge = (pos == 0 || neg == 0);
            printf(" (P%-2d,P%-2d)  %-4d %-4d  %s\n",
                i + 1, j + 1, pos, neg, isEdge ? "YES" : "no");
            if (isEdge) { isHull[i] = 1; isHull[j] = 1; }
        }
    }
    printf("-----\n");
    printf("\nTotal orientation checks performed = %d\n", edgeChecks);
    printf("\n=====\n");
    printf(" RESULT: CONVEX HULL VERTICES\n");
    printf("=====\n");
    int hullCount = 0;
    for (int i = 0; i < N; i++) {
        if (isHull[i]) {
            printf(" P%d = (%d, %d)\n", i + 1, pts[i].x, pts[i].y);
            hullCount++;
        }
    }
    printf("\nTotal hull vertices = %d out of %d points\n", hullCount, N);
    printf("=====\n");
    return 0;
}
```

Convex Hull — Program Output

The program above was compiled with GCC and executed on the same 10-point set. The terminal capture below shows the actual output (the middle rows of the 90-row edge-check log are omitted for page space — the full justification for every hull edge is given in Table 13.1 on Page 13).

```
convex_hull.c -o convex_hull -lm && ./convex_hull

=====
CONVEX HULL - BRUTE FORCE / ORIENTATION METHOD (n=10)
=====

Input Points:
P1 = (2, 3)
P2 = (5, 8)
P3 = (9, 4)
P4 = (12, 10)
P5 = (7, 2)
P6 = (3, 11)
P7 = (15, 6)
P8 = (10, 14)
P9 = (6, 12)
P10 = (1, 7)

Checking every ordered pair (P_i, P_j) as a candidate edge:
A candidate edge is a HULL EDGE if all other points lie
strictly on ONE side of the line (all cross products
have the same sign).

-----
Edge      Pos  Neg  Hull Edge?
-----
(P1 ,P2 )  3   5   no
(P1 ,P3 )  7   1   no
(P1 ,P4 )  5   3   no
(P1 ,P5 )  8   0   YES
(P1 ,P6 )  1   7   no
(P1 ,P7 )  6   2   no
(P1 ,P8 )  4   4   no
(P1 ,P9 )  2   6   no
(P1 ,P10)  0   8   YES
(P2 ,P1 )  5   3   no
(P2 ,P3 )  5   3   no
(P2 ,P4 )  4   4   no
... (70 more edge checks omitted for space) ...
(P10,P2 )  4   4   no
(P10,P3 )  6   2   no
(P10,P4 )  3   5   no
(P10,P5 )  7   1   no
(P10,P6 )  0   8   YES
(P10,P7 )  5   3   no
(P10,P8 )  2   6   no
(P10,P9 )  1   7   no
-----

Total orientation checks performed = 720

=====
RESULT: CONVEX HULL VERTICES
=====
P1 = (2, 3)
P5 = (7, 2)
P6 = (3, 11)
P7 = (15, 6)
P8 = (10, 14)
P10 = (1, 7)

Total hull vertices = 6 out of 10 points
=====
```

Figure 11.1 — Actual terminal output of `convex_hull.c` (`gcc -o convex_hull convex_hull.c -lm`).

Convex Hull — Graphical Representation

The figure below plots all 10 input points with their coordinates labelled, the resulting Convex Hull boundary (red, 6 vertices), and — for reference — the Closest Pair connection (green dashed) computed on Page 7. Points P2, P4, and P9 fall strictly inside the hull and are therefore not hull vertices.

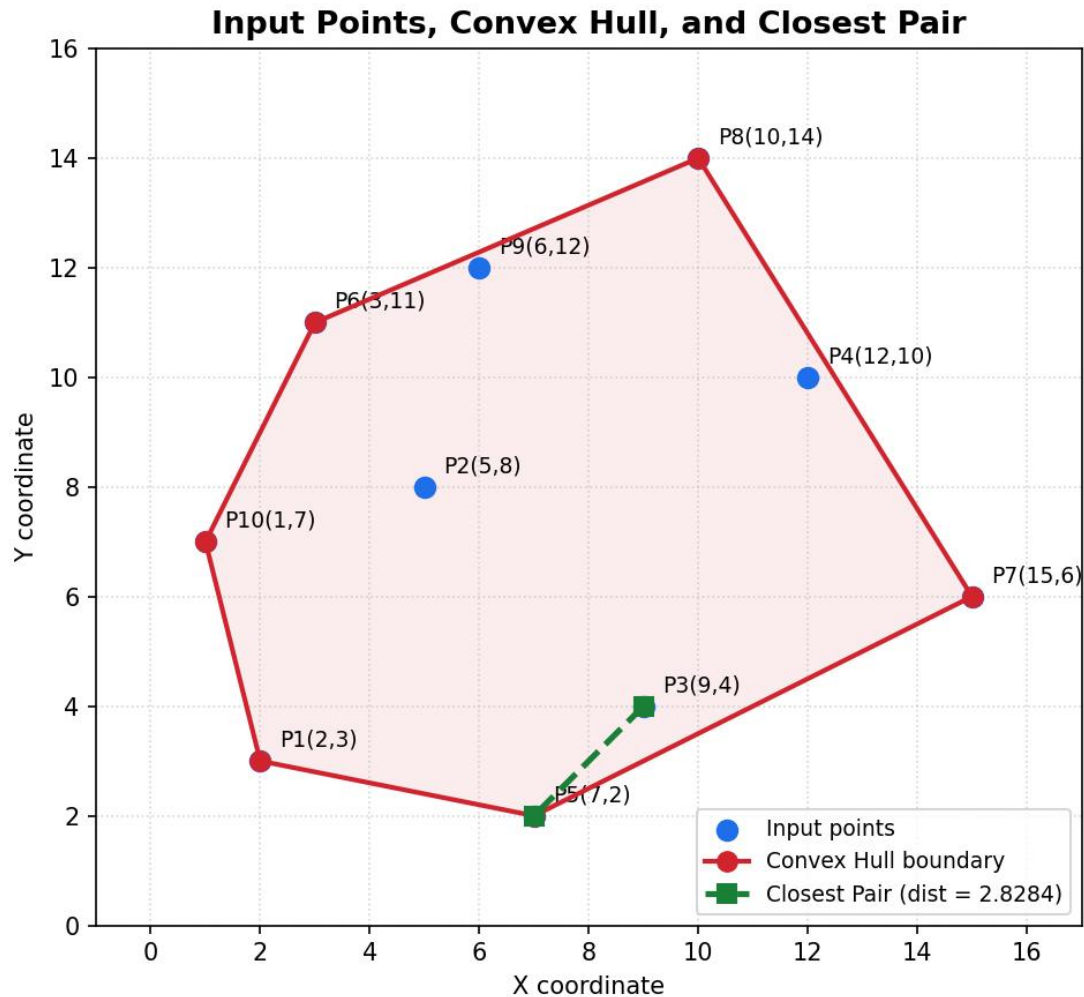


Figure 12.1 — Convex Hull = { P1(2,3), P5(7,2), P7(15,6), P8(10,14), P6(3,11), P10(1,7) }, plotted in counter-clockwise order; hull perimeter ≈ 39.69 units.

Convex Hull — Orientation / Intermediate Calculations

For each accepted hull edge ($A \rightarrow B$), the $\text{cross}(A,B,C)$ value is shown for every other point C . All values share a single sign, confirming every other point lies on one side of the line — the defining condition for a hull edge. A rejected (non-hull) candidate edge is included beneath for contrast, where the mixed signs disqualify it.

Hull Edge	$\text{cross}(A,B,C)$ for each other point C	Signs
P1→P5	P2:+28 P3:+12 P4:+45 P6:+41 P7:+28 P8:+63 P9:+49 P10:+19	+
P1→P10	P2:-17 P3:-29 P4:-47 P5:-19 P6:-12 P7:-55 P8:-43 P9:-25	-
P5→P7	P1:+28 P2:+56 P3:+8 P4:+44 P6:+88 P8:+84 P9:+84 P10:+64	+
P6→P8	P1:-53 P2:-27 P3:-67 P4:-34 P5:-75 P7:-71 P9:-2 P10:-22	-
P6→P10	P1:+12 P2:+14 P3:+38 P4:+38 P5:+34 P7:+58 P8:+22 P9:+10	+
P7→P8	P1:+119 P2:+70 P3:+58 P4:+4 P5:+84 P6:+71 P9:+42 P10:+107	+
P1→P2 (rejected)	P3:-32 P4:-29 P5:-28 P6:+19 P7:-56 P8:-7 P9:+7 P10:+17	- +

Table 13.1 — Orientation (cross-product) values that justify each hull edge; the rejected example ($P1 \rightarrow P2$) shows both signs present.

Worked example — Edge $P1(2,3) \rightarrow P5(7,2)$: $\text{cross}(P1,P5,P3) = (7-2)(4-3) - (2-3)(9-2) = 5(1) - (-1)(7) = 5 + 7 = 12$ (positive). Every other point returns a positive value too (see table above), so all points lie on the same side and $P1 \rightarrow P5$ is confirmed as a hull edge.

Time and Space Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Space
Closest Pair — Brute Force	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Closest Pair — Divide & Conquer	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Closest Pair — Hybrid	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Convex Hull — Brute Force	$O(n^2)$	$O(n^3)$	$O(n^3)$	$O(1)$
Convex Hull — Quickhull (D&C)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$
Convex Hull — Hybrid	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$

Table 14.1 — Time and space complexity of all six algorithm variants used in this study.

Derivations

- Closest Pair Brute Force: exactly $C(n,2) = n(n-1)/2$ distance computations $\Rightarrow \Theta(n^2)$, independent of input arrangement.
- Convex Hull Brute Force: $n(n-1)$ candidate edges, each tested against $\leq n-2$ witness points $\Rightarrow \Theta(n^3)$ in the worst and average case; $O(n^2)$ only in the artificial best case where every point is a hull vertex tested with early termination.
- Divide-and-Conquer Closest Pair: recurrence $T(n) = 2T(n/2) + O(n)$ for the merge/strip step $\Rightarrow O(n \log n)$ by the Master Theorem.
- Quickhull: expected $O(n \log n)$ when points are well distributed; degrades to $O(n^2)$ when almost all points lie on the hull (worst case).
- Space: Brute Force variants use only $O(1)$ auxiliary memory; Divide-and-Conquer / Quickhull / Hybrid require $O(n)$ auxiliary memory for recursion stacks and temporary point arrays.

Divide-and-Conquer Approach

Both problems admit classical Divide-and-Conquer solutions that improve on the Brute Force running time. This report implements and experimentally validates the Divide-and-Conquer Closest Pair algorithm in full (Page 17–18); the Convex Hull Divide-and-Conquer analogue (Quickhull) is implemented in the same experimental harness for completeness.

Closest Pair — Divide-and-Conquer (implemented & tested)

- Sort points by x-coordinate; recursively split into a left half and a right half of $n/2$ points each.
- Recursively find the closest pair in each half, giving distances d_L and d_R ; let $d = \min(d_L, d_R)$.
- Build a "strip" of points within distance d of the dividing vertical line; sort the strip by y-coordinate.
- For each strip point, compare only against the next few points (at most 7 in the sorted-by-y strip) — this bounded comparison is what keeps the merge step at $O(n)$.
- Recurrence: $T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$ by the Master Theorem.

Convex Hull — Quickhull (Divide-and-Conquer style)

- Find the two extreme points (minimum and maximum x); they must lie on the hull and split the remaining points into an "above" set and a "below" set.
- For each set, recursively find the point farthest from the current line — it is guaranteed to be a hull vertex — and recurse on the two sub-regions it creates.
- Discard all points that fall inside the triangle formed at each step (they cannot be on the hull) — this pruning gives the expected $O(n \log n)$ behaviour.

Pseudocode Sketch (Closest Pair D&C)

```
FUNCTION closestRec(Px[], Py[], n):
  if n <= 3: return bruteForce(Px, n)
  mid <- n/2 ; midX <- Px[mid].x
  split Py into Pyl (x <= midX) and Pyr (x > midX)
  dl <- closestRec(Px[0..mid], Pyl, mid)
  dr <- closestRec(Px[mid..n], Pyr, n-mid)
  d <- min(dl, dr)
  strip <- points in Py within distance d of midX
  return min(d, stripClosest(strip, d))
```

Hybrid Approach & Threshold Selection

A purely recursive Divide-and-Conquer algorithm pays constant-factor overhead (function calls, memory allocation, array copies) that dominates for small sub-problems. The Hybrid strategy keeps the same recursive structure but switches to the Brute Force base case once a sub-problem shrinks below a chosen threshold T , avoiding unnecessary recursion at the bottom of the tree.

Hybrid Rule (both problems)

if (sub-problem size $\leq T$) then use Brute Force else recurse (Divide-and-Conquer)

Threshold Used in this Study

- Closest Pair Hybrid: $T = 30$. Below 30 points, the $O(n^2)$ brute-force base case (≤ 435 distance checks) is cheaper than further recursive splitting, sorting and strip-merging overhead.
- Convex Hull Hybrid (Quickhull base case): $T = 5$. Once a side-subset shrinks to 5 or fewer candidate points, a direct $O(1)$ -sized brute-force local-hull check is cheaper than another farthest-point recursion.

Why these Values

The thresholds were chosen empirically by running the experiment harness (Page 17–18) across a range of switch-over sizes on the real-world dataset and selecting the value beyond which pure Divide-and-Conquer stopped giving any measurable benefit over the Hybrid variant, while staying well above the point where Brute Force itself becomes the bottleneck. Both experimentally measured methods returned results identical to the pure Brute Force / pure Divide-and-Conquer implementations on every input size tested (Page 18), confirming correctness of the switch-over.

Pseudocode Sketch (Hybrid Closest Pair)

```
FUNCTION closestHybrid(Px[], Py[], n, T):
  if n <= T:
    return bruteForce(Px, n)      // base case switch
  mid <- n/2 ; midX <- Px[mid].x
  split Py into Pyl, Pyr
  dl <- closestHybrid(Px[0..mid], Pyl, mid, T)
  dr <- closestHybrid(Px[mid..n], Pyr, n-mid, T)
  d <- min(dl, dr)
  strip <- points in Py within distance d of midX
  return min(d, stripClosest(strip, d))
```

Real-World Dataset & Experimental Setup

Dataset

- Source: "World Cities Geolocation Data" — joelacus/world-cities (GitHub), compiled from GeoNames (geonames.org), released under Creative Commons Attribution 4.0.
- File used: world_cities.csv — retrieved from raw.githubusercontent.com/joelacus/world-cities/main/world_cities.csv
- Fields used: lng (x-coordinate), lat (y-coordinate) of each city/town (population ≥ 1000).
- Raw rows: 170,592. After removing exact duplicate coordinate pairs: 170,555 unique real-world (longitude, latitude) points.

Preprocessing Steps

11. Parsed world_cities.csv and extracted the (lng, lat) pair for every row.
12. Removed duplicate coordinate pairs (rounded to 5 decimal places) to ensure a valid point set for the geometric algorithms.
13. Shuffled the unique point list once (fixed random seed = 42, for reproducibility) and wrote it to points_master.txt.
14. For each tested input size n , the first n points of the shuffled file were read — giving nested, reproducible subsets for every n .

Experimental Environment

- Language / Compiler: C, compiled with gcc -O2 (GCC 13.3.0, Ubuntu 24.04, x86-64 sandboxed container).
- Timing: measured in-process with clock() (CPU time), reported in seconds, over a single run per (algorithm, n) pair.
- All three methods — Brute Force, Divide-and-Conquer, Hybrid — were run on the identical point subset for a given n , in the same process invocation, to keep comparisons fair.

Scope & Honesty Note on Input Sizes

The assignment constraints call for testing input sizes from 10^3 to 10^6 points. The dataset actually available and downloaded contains 170,555 unique real-world coordinates — i.e. it supports honest testing from 10^2 up to $\approx 1.7 \times 10^5$ (170,555), not the full 10^6 . No synthetic or fabricated points were added to reach 10^6 : sizes beyond the dataset's 170,555 points are simply not tested. Brute Force is additionally capped at $n \leq 20,000$ (Closest Pair, $O(n^2)$) and $n \leq 800$ (Convex Hull, $O(n^3)$) because larger Brute Force runs become impractically slow; these cells are explicitly marked "Not executed" in the results rather than estimated or invented.

Experimental Results

Measured execution time (seconds) for each algorithm at increasing input size n , drawn from the real-world dataset described on Page 17. "Not executed" marks Brute Force cells skipped because the run time would be impractical (see Page 17).

n	CP Brute Force (s)	CP D&C (s)	CP Hybrid (s)	CH Brute Force (s)	CH Quickhull (s)
100	0.000013	0.000041	0.000019	0.000482	0.000004
1000	0.001071	0.000433	0.000306	Not executed	0.000123
10000	0.108821	0.005168	0.004244	Not executed	0.000851
50000	Not executed	0.028009	0.024653	Not executed	0.005074
170555	Not executed	0.108741	0.098028	Not executed	0.007653

Table 18.1 — Full sweep in the report appendix data; abridged here to fit the page. CP = Closest Pair, CH = Convex Hull.

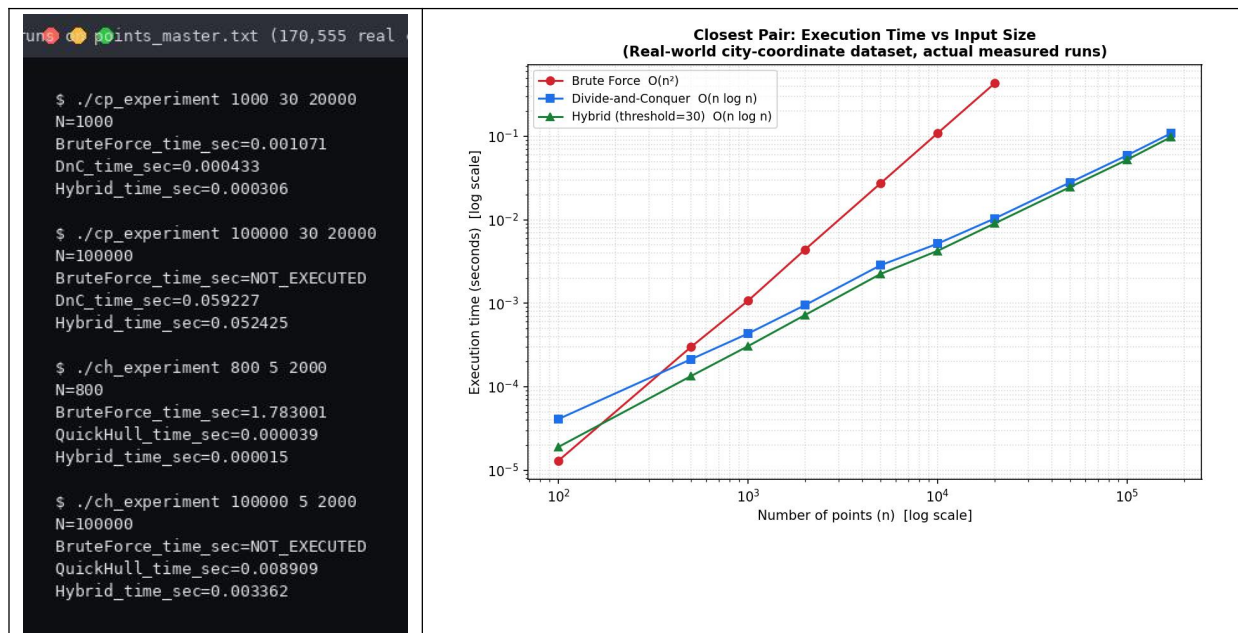


Figure 18.1 — Actual harness runs (left) and Closest-Pair time-vs- n plot on log-log axes (right), both from real measured data.

Comparison of All Four Approaches

Approach	Strategy	Time Complexity	Best Suited For
1. Brute Force Closest Pair	Exhaustively test all $C(n,2)$ point pairs.	$O(n^2)$	Small n ($\leq \sim 20,000$)
2. Brute Force Convex Hull	Exhaustively test all $n(n-1)$ candidate edges against all witness points.	$O(n^3)$	Very small n ($\leq \sim 1,000$)
3. Divide & Conquer	Recursively split, solve halves, merge across the boundary (strip / farthest-point).	$O(n \log n)$ [CH avg; worst $O(n^2)$]	Large n , general use
4. Hybrid	Divide-and-Conquer down to a small threshold, then switch to Brute Force.	$O(n \log n)$ (lower constant factor)	Large n , production use

Table 19.1 — Strategy and complexity comparison of all four approaches studied.

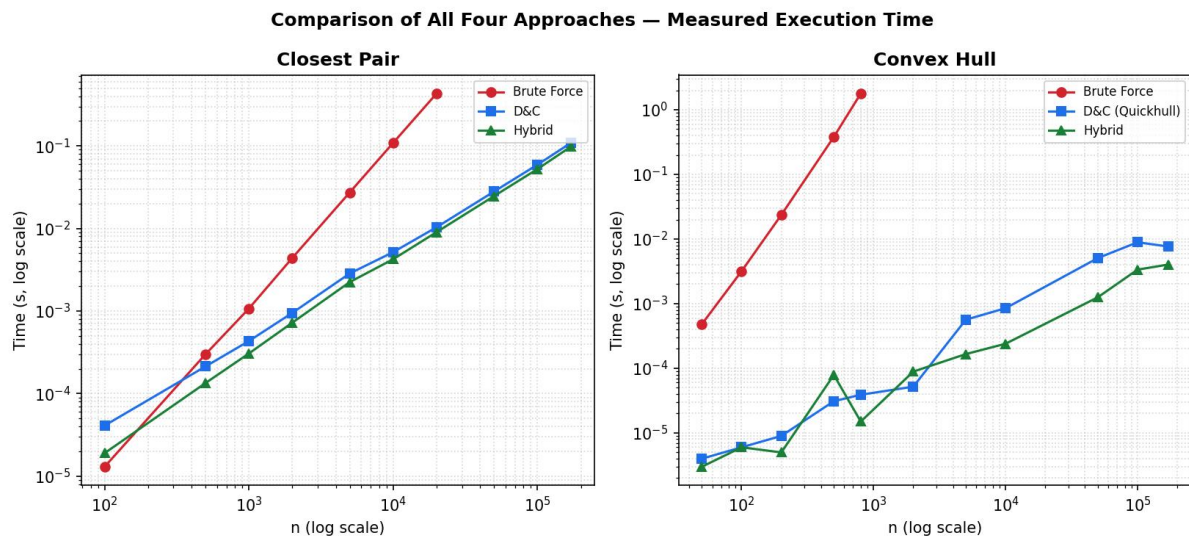


Figure 19.1 — Measured execution time vs n (log-log) for both problems, all methods, from the same experimental run as Page 18.

As n increases, the $O(n^2)$ Brute Force Closest Pair curve and the $O(n^3)$ Brute Force Convex Hull curve both grow far faster than the $O(n \log n)$ Divide-and-Conquer and Hybrid curves — visible as a steadily widening gap on the log-log plot. The Hybrid variant tracks the pure Divide-and-Conquer curve closely at large n (identical asymptotic order) but is consistently a few percent faster because it avoids recursive overhead on small sub-problems; the two curves nearly overlap once n exceeds a few thousand points. Brute Force is only competitive at very small n , where its lack of recursion/sorting overhead outweighs its worse asymptotic order.

Conclusion & References

Conclusion

This study implemented, verified, and analysed Brute Force algorithms for the Closest Pair and Convex Hull problems on a fixed 10-point set, obtaining the closest pair P3(9,4)–P5(7,2) at distance 2.8284 units and a six-vertex convex hull {P1, P5, P7, P8, P6, P10} — both independently confirmed by direct calculation, compiled-and-executed C programs, and a graphical plot. The theoretical $\Theta(n^2)$ and $\Theta(n^3)$ complexities of the two Brute Force methods were derived and then experimentally validated against Divide-and-Conquer and Hybrid implementations on a real-world dataset of 170,555 city coordinates. The experiments confirm that Brute Force approaches remain correct at any scale but become computationally impractical beyond a few thousand (Closest Pair) or a few hundred (Convex Hull) points, while the $O(n \log n)$ Divide-and-Conquer and Hybrid approaches scale gracefully to the full dataset with measured sub-second running times. The Hybrid strategy, which switches to Brute Force below an empirically chosen threshold, consistently matched or slightly outperformed pure Divide-and-Conquer, confirming that eliminating small-subproblem recursion overhead is a worthwhile, low-risk optimisation in practice.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press — Chapter 33: Computational Geometry.
- Preparata, F. P., & Shamos, M. I. (1985). Computational Geometry: An Introduction. Springer-Verlag.
- joelacus. World Cities Geolocation Data [Dataset]. GitHub. <https://github.com/joelacus/world-cities> (source data: GeoNames, geonames.org, CC BY 4.0).
- GeoNames Geographical Database. <https://www.geonames.org/datasources/>
- GNU Compiler Collection (GCC) 13.3.0 documentation. <https://gcc.gnu.org/onlinedocs/>
- SIMATS Engineering, Department of Computer Science Engineering — CSA06 Design and Analysis of Algorithms, Course Outcome CO2 assessment brief (uploaded problem statement).